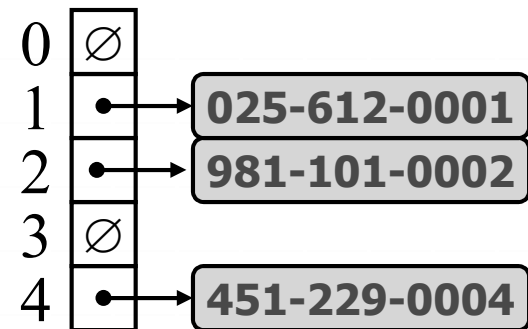


Hash Tables



Hash Functions and Hash Tables (§ 8.2)

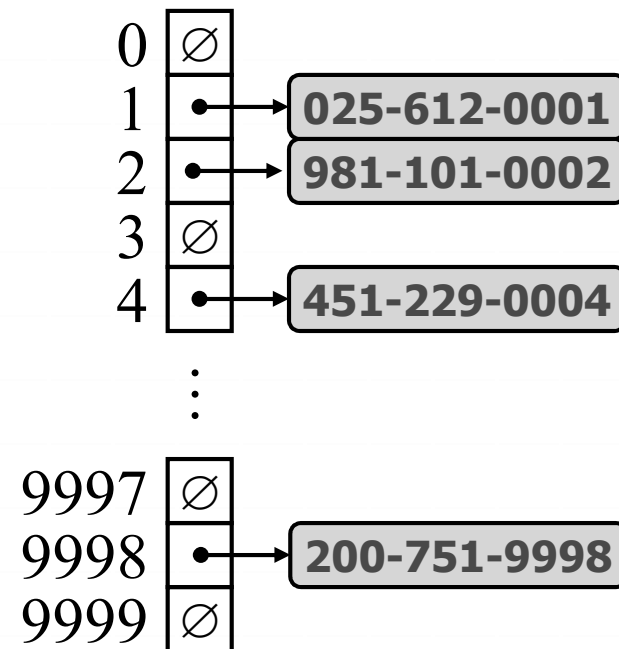


- ◆ A hash function h maps keys of a given type to integers in a fixed interval $[0, N - 1]$
- ◆ Example:
$$h(x) = x \bmod N$$

is a hash function for integer keys
- ◆ The integer $h(x)$ is called the hash value of key x
- ◆ A hash table for a given key type consists of
 - Hash function h
 - Array (called table) of size N
- ◆ When implementing a map with a hash table, the goal is to store item (k, o) at index $i = h(k)$

Example

- ◆ We design a hash table for a map storing entries as (SSN, Name), where SSN (social security number) is a nine-digit positive integer
- ◆ Our hash table uses an array of size $N = 10,000$ and the hash function $h(x) = \text{last four digits of } x$



Hash Functions (§ 8.2.2)



- ◆ A hash function is usually specified as the composition of two functions:

Hash code:

$h_1: \text{keys} \rightarrow \text{integers}$

Compression function:

$h_2: \text{integers} \rightarrow [0, N - 1]$

- ◆ The hash code is applied first, and the compression function is applied next on the result, i.e.,

$$h(x) = h_2(h_1(x))$$

- ◆ The goal of the hash function is to “disperse” the keys in an apparently random way

Compression Functions

(§ 8.2.4)



◆ Division:

- $h_2(y) = y \bmod N$
- The size N of the hash table is usually chosen to be a prime
- The reason has to do with number theory and is beyond the scope of this course

◆ Multiply, Add and Divide (MAD):

- $h_2(y) = (ay + b) \bmod N$
- a and b are nonnegative integers such that
$$a \bmod N \neq 0$$
- Otherwise, every integer would map to the same value b

Collision Handling

(§ 8.2.5)



- ◆ Collisions occur when different elements are mapped to the same cell



- ◆ **Separate Chaining:**
let each cell in the table point to a linked list of entries that map there

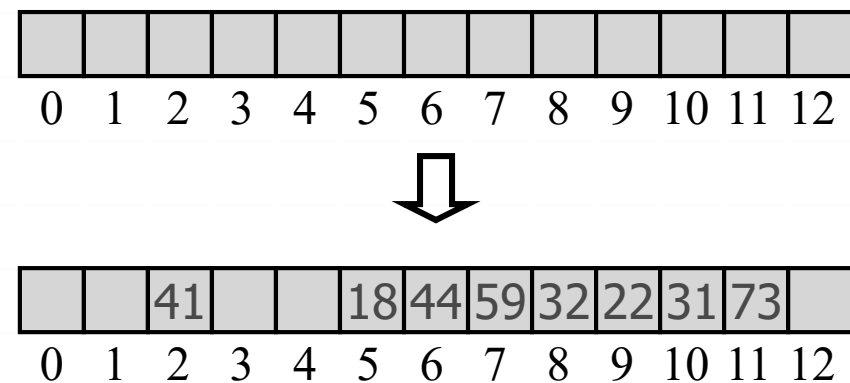
- ◆ Separate chaining is simple, but requires additional memory outside the table

Linear Probing

- ◆ Open addressing: the colliding item is placed in a different cell of the table
- ◆ **Linear probing** handles collisions by placing the colliding item in the next (circularly) available table cell
- ◆ Each table cell inspected is referred to as a “probe”
- ◆ Colliding items lump together, causing future collisions to cause a longer sequence of probes

◆ Example:

- $h(x) = x \bmod 13$
- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order



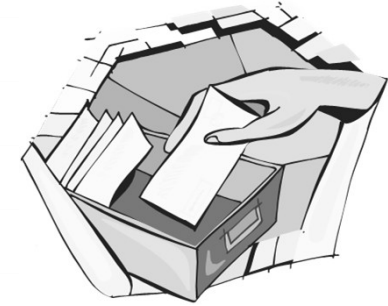
Search with Linear Probing



- ◆ Consider a hash table A that uses linear probing
- ◆ $\text{get}(k)$
 - We start at cell $h(k)$
 - We probe consecutive locations until one of the following occurs
 - ◆ An item with key k is found, or
 - ◆ An empty cell is found, or
 - ◆ N cells have been unsuccessfully probed

```
Algorithm  $\text{get}(k)$   
   $i \leftarrow h(k)$   
   $p \leftarrow 0$   
  repeat  
     $c \leftarrow A[i]$   
    if  $c = \emptyset$   
      return null  
    else if  $c.\text{key}() = k$   
      return  $c.\text{element}()$   
    else  
       $i \leftarrow (i + 1) \bmod N$   
       $p \leftarrow p + 1$   
  until  $p = N$   
  return null
```


Double Hashing



- ◆ Double hashing uses a secondary hash function $d(k)$ and handles collisions by placing an item in the first available cell of the series
 $(i + jd(k)) \bmod N$
for $j = 0, 1, \dots, N - 1$
- ◆ The secondary hash function $d(k)$ cannot have zero values
- ◆ The table size N must be a prime to allow probing of all the cells

- ◆ Common choice of compression function for the secondary hash function:

$$d_2(k) = q - k \bmod q$$

where

- $q < N$
- q is a prime
- ◆ The possible values for $d_2(k)$ are
 $1, 2, \dots, q$

Example of Double Hashing

- ◆ Consider a hash table storing integer keys that handles collision with double hashing

- $N = 13$
- $h(k) = k \bmod 13$
- $d(k) = 7 - k \bmod 7$

- ◆ Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

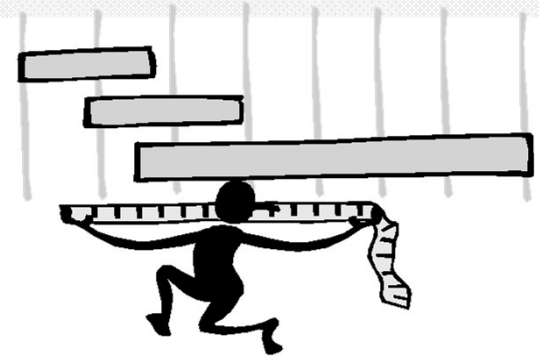
k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

0	1	2	3	4	5	6	7	8	9	10	11	12



31		41			18	32	59	73	22	44		
0	1	2	3	4	5	6	7	8	9	10	11	12

Performance of Hashing



- ◆ In the worst case, searches, insertions and removals on a hash table take $O(n)$ time
- ◆ The worst case occurs when all the keys inserted into the map collide
- ◆ The load factor $\alpha = n/N$ affects the performance of a hash table
- ◆ The expected running time of all the dictionary ADT operations in a hash table is $O(1)$
- ◆ In practice, hashing is very fast provided the load factor is not close to 100%
- ◆ Applications of hash tables:
 - small databases
 - compilers
 - browser caches